



Taller de Programación Literaria en Python con Emacs

Jami Tonato Mateo Israel

2026-06-02

1 Objetivos

- Familiarizar al estudiante con el uso de Python dentro de Org mode en Emacs.
- Practicar programación literaria combinando texto, matemáticas y código ejecutable.
- Usar `C-c C-c` para ejecutar bloques y `C-c '` para editar bloques en un buffer especializado.
- Resolver ejercicios sencillos de Python inspirados en temas frecuentes de cheat sheets: variables, tipos, listas, condicionales, bucles y funciones.

2 Instrucciones Generales

1. Realice todas las actividades en Linux o WSL.
2. Abra este archivo en Emacs y guárdelo con un nombre como `Taller-Python-Literario-(Apellido 0 Grupo).org`.
3. Ejecute bloques de código con `C-c C-c` cuando el cursor esté dentro del bloque. Org mode ejecuta el bloque y coloca el resultado en una sección `#+RESULTS`.
4. Para editar cómodamente un bloque de Python use `C-c '`. Esto abre un buffer de edición con el modo principal apropiado para el lenguaje; al usar `C-c '` nuevamente regresa al documento Org, y con `C-x C-s` se actualiza el contenido del bloque en el archivo base.
5. En este taller, en lugar de modificar código ya terminado, usted debe **escribir la lógica de programa** dentro de los bloques propuestos.
6. Exporte el documento a PDF con `M-x org-latex-export-to-pdf` cuando haya terminado.
7. Suba al aula virtual el archivo `.org` y el `.pdf`.

3 Recursos Adicionales

Puede consultar los siguientes recursos en Internet y en el Repositorio Git de la clase

- Este trabajo
- Tutorial de Python y Java en Emacs
- Tutorial $\text{\LaTeX}$ Python e Emacs
- The only Python cheatsheet you will ever need
- Comprehensive Python Cheatsheet
- Capítulo 2 del Libro *Practical Computer Architecture with Python and ARM*[1]
- Working with Source Blocks in Emacs

4 Cómo trabajar con bloques de Python

4.1 Flujo recomendado

1. Ubique el cursor dentro de un bloque `#+begin_src python`.
2. Presione `C-c '` para abrir el bloque en una zona de edición más cómoda con soporte del modo de Python en Emacs.
3. Escriba la lógica del programa dentro de ese buffer.
4. Guarde con `C-x C-s` para actualizar el bloque en el archivo Org.
5. Regrese con `C-c '` al documento principal.
6. Ejecute el bloque con `C-c C-c` para observar el resultado insertado por Org mode.

4.2 ¿Qué debe resolver el estudiante?

En cada actividad encontrará uno o más bloques de Python con comentarios que indican el objetivo. Su tarea consiste en:

- Leer la explicación del tutorial.
- Escribir el código solicitado dentro del bloque.
- Ejecutarlo con `C-c C-c`.
- Verificar la salida.
- Explicar brevemente lo observado cuando se le pida.

No se espera copiar código de memoria. Se espera construir soluciones pequeñas y entender la relación entre texto, código y resultados.

5 Parte 1: Tutorial guiado de Python en Org mode

Esta primera parte es de familiarización. El objetivo es practicar conceptos básicos de Python y usar repetidamente `C-c '` y `C-c C-c`.

5.1 Variables y tipos

Python permite almacenar valores en variables de diferentes tipos como enteros, flotantes y cadenas. Estos son temas introductorios comunes en hojas de referencia para principiantes.

5.1.1 Actividad 1

Escriba un programa que:

- Declare una variable entera llamada `edad`.
- Declare una variable flotante llamada `promedio`.
- Declare una cadena llamada `nombre`.
- Imprima el valor y el tipo de cada variable.

```
# Escriba aquí su programa
edad = 18
promedio = 9.89
nombre = "Mateo"
print(edad,type(edad))
print(promedio,type(promedio))
print(nombre,type(nombre))
# Debe usar variables: edad, promedio, nombre

18 <class 'int'>
9.89 <class 'float'>
Mateo <class 'str'>
```

Explique en 2 líneas qué diferencia observa entre `int`, `float` y `str`: Para declarar una variable entera, no se debe colocar el `'.'` decimal, ya que si no, python lo va a tomar como un flotante y cuando se declara una variable tipo `'str'` se debe colocar después del igual, la cadena entre comillas o comillas simples. De esta forma se diferencian los tres tipos de variables primitivas que pedía el ejercicio.

5.2 Operaciones y expresiones

Python permite realizar operaciones aritméticas como suma, producto, potencia y residuo.

5.2.1 Actividad 2

Escriba un programa que:

- Defina dos variables numéricas `a` y `b`.
- Imprima su suma, resta, multiplicación, división y residuo.

```
# Escriba aquí su programa
# Primero se definen los números a y b
a = 2
b = 6
# Imprimimos los parámetros solicitados
print("Suma = ", a+b)
print("Resta = ", a-b)
print("Multiplicación = ", a*b)
print("División = ", a/b)
print("Módulo = ", a%b)

Suma = 8
Resta = -4
Multiplicación = 12
División = 0.3333333333333333
Módulo = 2
```

Escriba una línea indicando para qué sirve el operador `%`: El operador `%` sirve para obtener el módulo o residuo de un número, que es resultado de una división de enteros.

5.3 Listas

Las listas permiten agrupar varios valores y recorrerlos. Las hojas de referencia introductorias suelen mostrar creación, acceso, modificación y recorrido de listas.

5.3.1 Actividad 3

Escriba un programa que:

- Cree una lista llamada `numeros` con cinco enteros.
- Imprima la lista completa.
- Imprima el primer y el último elemento.
- Imprima la longitud de la lista.

```
# Escriba aquí su programa
# Primero definimos la lista
numeros = [12,18,73,23,19]
# Después operamos los items solicitados
print("La lista es: ", numeros)
print("El primer número es: ", numeros[0])
print("El último número es: ", numeros[-1])
print("La longitud de la lista es: ", len(numeros))
```

```
La lista es: [12, 18, 73, 23, 19]
El primer número es: 12
El último número es: 19
La longitud de la lista es: 5
```

Explique en 1 o 2 líneas qué hace `len()`: El método `len()` sirve para calcular la longitud, el tamaño o el número de elementos de una lista u objeto.

5.4 Bucles y condicionales

Los bucles permiten repetir acciones y los condicionales permiten tomar decisiones según una condición lógica.

5.4.1 Actividad 4

Escriba un programa que:

- Recorra los números del 1 al 10.
- Imprima cuáles son pares y cuáles son impares.

```
# Escriba aquí su programa
# Primero se va a recorrer los números del 1 al 10
for i in range(1,11):
    if i%2 == 0:
        print("El número", i, "es par")
    else:
        print("El número", i, "es impar")
#Aquí se termina el bloque for
```

```
El número 1 es impar
El número 2 es par
El número 3 es impar
El número 4 es par
El número 5 es impar
El número 6 es par
```

```
El número 7 es impar
El número 8 es par
El número 9 es impar
El número 10 es par
```

Describa brevemente cómo usó `if` y el operador `%`: El condicional `if` se usó para determinar si el número a comparar era par o impar con ayuda del módulo `%` para saber el residuo cuando se dividió para 2.

5.5 Comprensión de listas

La comprensión de listas es una manera concisa de crear listas en Python.

5.5.1 Actividad 5

Escriba un programa que:

- Genere una lista con los números del 1 al 10.
- Genere otra lista con los cuadrados de esos números usando **list comprehension**.
- Imprima ambas listas.

```
# Escriba aquí su programa
# Primero generamos la lista con los números del 1 al 10
numeros = list(range(1,11))
cuadrados = [x**2 for x in numeros]
# Ahora imprimimos las listas
print("La lista de números del 1 al 10 es: ", numeros)
print("La lista de los cuadrados de la lista de números es: ", cuadrados)
```

La lista de números del 1 al 10 es: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

La lista de los cuadrados de la lista de números es: [1, 4, 9, 16, 25, 36, 49, 64, 81, 100]

Explique en 2 líneas qué ventaja tiene una **list comprehension** frente a escribir varios `append`: Cuando definimos una lista por comprensión, en esta se pueden tener varios elementos que se definen por la regla establecida, mientras que por notación o `append` se debe colocar manualmente los datos, lo que dificulta tener listas grandes o con valores que se relacionen entre sí.

5.6 Funciones

Las funciones permiten encapsular lógica y reutilizar código; son una parte básica y frecuente en resúmenes de Python.

5.6.1 Actividad 6

Escriba un programa que:

- Defina una función `cuadrado(x)`.
- La función debe retornar el cuadrado de `x`.
- Luego pruebe la función con 3 valores e imprima los resultados.

```
# Escriba aquí su programa
# Vamos a definir la función cuadrado de x
def cuadrado(x):
    return x**2
# Ahora vamos a probar la función con 3 valores
print("El cuadrado de 4 es: ", cuadrado(4))
print("El cuadrado de 8 es: ", cuadrado(8))
print("El cuadrado de 5 es: ", cuadrado(5))

El cuadrado de 4 es:  16
El cuadrado de 8 es:  64
El cuadrado de 5 es:  25
```

Escriba en 2 líneas por qué las funciones ayudan a organizar programas: Las funciones en la programación en general, ayudan a dividir grandes problemas en módulos o pasos mas pequeños y estructurales, por ello, su uso es de vital importancia en la resolución de problemas.

5.7 Texto, matemáticas y código

Una ventaja de la programación literaria es documentar una idea con texto y validarla con código ejecutable en el mismo archivo.[cite:4]

Considere la fórmula de la suma de los primeros n números naturales:

$$S(n) = \frac{n(n+1)}{2}$$

5.7.1 Actividad 7

Escriba un programa que:

- Calcule la suma de 1 hasta n usando un bucle.
- Calcule la misma suma usando la fórmula.
- Compare los resultados para $n = 10$.

```
# Escriba aquí su programa
n = 10;
# Primero se va a calcular la suma acumulada con un bucle
sumaBucle = 0
for i in range(1,n+1):
    sumaBucle += i
# Ahora vamos a calcular la suma acumulada con la fórmula
sumaFormula = (n*(n+1))//2
# Comparar e imprimir los resultados
print("Resultado con bucle:", sumaBucle)
print("Resultado con fórmula:", sumaFormula)
print("¿Son iguales los resultados?:", sumaBucle == sumaFormula)

Resultado con bucle: 55
Resultado con fórmula: 55
¿Son iguales los resultados?: True
```

Explique en 3 líneas cómo este ejemplo muestra la idea de programación literaria: Este ejemplo logra explicar muy bien la programación literaria porque se puede llegar a la soluciones matemáticas a través de la utilización de sus fórmulas y también con razonamientos propios con el uso de diferentes datos y estructuras en el código.

6 Parte 2: Resolución de problemas sencillos

En esta segunda parte ya no se explica el concepto paso a paso. Ahora debe crear los bloques de código resolviendo pequeños problemas. Se recomienda usar `C-c` para editar cada bloque y `C-c C-c` para probar la solución.

6.1 Problema 1

Escriba un programa que lea una lista fija de calificaciones [7, 8, 10, 9, 6], calcule el promedio y lo imprima.

```
# Escriba aquí su solución
# Primero definimos la función que recibirá como parámetro una lista y calculará el promedio
def calcularPromedio(lista):
    suma = 0
    for nota in lista: # Itera sobre los elementos de la lista, en este caso, notas
        suma += nota
    return suma / len(lista)

# Creamos la lista y llamamos a la función
calificaciones = [7, 8, 10, 9, 6]
resultado = calcularPromedio(calificaciones)
print("El promedio de calificaciones es:", resultado)
```

El promedio de calificaciones es: 8.0

6.2 Problema 2

Escriba un programa que cuente cuántas vocales hay en la cadena "arquitectura".

```
# Escriba aquí su solución
# Primero definimos la lista y el contador
cadena = "arquitectura"
contadorVocales = 0
#Después recorreremos toda la cadena
for letra in cadena:
    # Comprobamos si la letra es una vocal
    if letra in "aeiou": # Esta es una forma de representar todas las opciones posibles a comparar
        contadorVocales += 1
# Imprimimos el resultado
print("El número total de vocales es:", contadorVocales)
```

El número total de vocales es: 6

6.3 Problema 3

Escriba un programa que construya una lista con los números pares del 2 al 20.

```
# Escriba aquí su solución
# Definimos la lista con la condición dada
numeros = list(range(2,21))
pares = [x for x in numeros if x%2==0]
print("La lista con los numeros pares entre el 2 al 20 es: ", pares)
```

La lista con los numeros pares entre el 2 al 20 es: [2, 4, 6, 8, 10, 12, 14, 16, 18, 20]

6.4 Problema 4

Escriba una función `maximo_de_tres(a, b, c)` que retorne el mayor de tres números y pruébela con un ejemplo.

```
# Escriba aquí su solución
# Definimos la función que calculará el máximo de 3 números
def maximoDeTres(a,b,c):
    return max(a,b,c)
# Con el método max() de python, se calculará el maximo entre los
# números o parámetros comparados.
max = maximoDeTres(45,67,89)
print("El máximo de los tres números es de: ", max)
```

El máximo de los tres números es de: 89

6.5 Problema 5

Escriba un programa que imprima la tabla de multiplicar del 5, desde 5 x 1 hasta 5 x 10.

```
# Escriba aquí su solución
# Definimos la función que imprimirá la tabla de multiplicar de un número cualquiera
def imprimirTabla(numeros, num):
    for numero in numeros:
        resultado = num*numero
        print(num, " x ", numero, " = ", resultado)
# Definir una lista con los números del 1 al 10
numeros = list(range(1,11))
imprimirTabla(numeros, 5)
```

```
5 x 1 = 5
5 x 2 = 10
5 x 3 = 15
5 x 4 = 20
5 x 5 = 25
5 x 6 = 30
5 x 7 = 35
5 x 8 = 40
5 x 9 = 45
5 x 10 = 50
```

6.6 Problema 6

Escriba un programa que determine si la palabra "reconocer" es palíndroma.

```
# Escriba aquí su solución
palabra = "reconocer"
palabraInvertida = ""
# Recorremos el texto desde el último carácter hasta el primero
# range(inicio, fin, paso)
# Empezamos en la posición (longitud - 1) y vamos restando 1 hasta llegar a 0
for i in range(len(palabra) - 1, -1, -1):
    palabraInvertida += palabra[i]
# Condicional para verificar si la palabra original y la auxiliar son iguales
```



```

if palabra == palabraInvertida:
    print("La palabra '" + palabra + "' es palíndroma")
else:
    print("La palabra '" + palabra + "' no es palíndroma")

```

La palabra 'reconocer' es palíndroma

7 Equipo de Trabajo

Complete la información de los integrantes del grupo e indique el líder.

Nombre	email	Rol
Jami Tonato Mateo Israel	mateo.jami@epn.edu.ec	líder

8 Verificación de Entregables [100%]

- ☒ Parte 1 completada.
- ☒ Parte 2 completada.
- ☒ Cuestionario de 10 preguntas respondido en el aula virtual.
- ☒ Uso de C-c ' para editar bloques practicado.
- ☒ Ejecución de bloques con C-c C-c verificada.
- ☒ Archivo PDF generado con M-x org-latex-export-to-pdf.
- ☒ Entrega de archivo .org y .pdf.

8.1 Problemas con la Tarea / Comentarios

- Actividad no completada debido a:
- Dificultades encontradas con Emacs, Org mode o Python:
- Comentarios adicionales:

9 Referencias

References

- [1] Alan Clements. *Practical Computer Architecture with Python and ARM: An introductory guide for enthusiasts and students to learn how computers work and program their own*. Birmingham: Packt Publishing, 2023. ISBN: 9781837636679.